Part 1 – Python and Object-Oriented Programming

Q1. What are the entities in our academic model?

Answer:

The main entities in our academic model are Semester, Teaching Unit (Unit / UE), Module, and Student.

A module contains a code or name, a title, a coefficient, credits, weekly lecture/TD/TP hours, teaching mode, evaluation percentages, and grades. A unit contains modules, and a semester contains units. A student receives grades and accumulates credits.

Q2. Create and test the Module class.

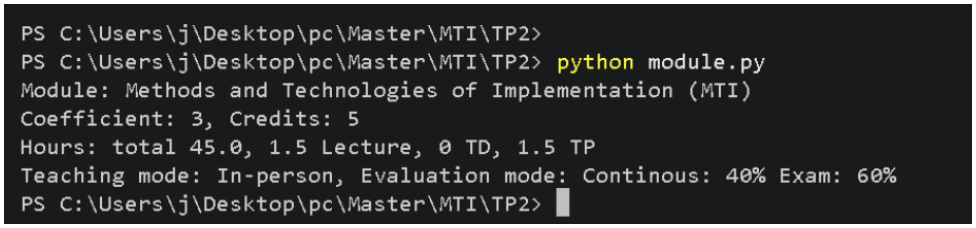
Answer:

The `Module` class stores module information, calculates the total semester workload as

$$15 \times (\text{lecture} + \text{TD} + \text{TP}),$$

stores grades, prints a summary, and provides methods for calculating the average and credits.

Screenshot :



Q3. Implement `calculate_average()` and `calculate_credits()`.

Answer:

The continuous-assessment percentage is split equally between TP and TD when both exist. If only one exists, it receives the full continuous-assessment percentage. The exam receives the examination percentage.

Credits are awarded if the module average is at least 10/20. Otherwise, the module earns zero credits.

Screenshot (optional):

Q4. Implement the Unit class.

Answer:

The `Unit` class inherits from `AcademicElement` and stores a collection of `Module` objects. The method `add_module(module)` adds a module to that collection.

Code link:

Q5. Identify similarities between Unit and Module.

Answer:

Both are academic elements and share common attributes, such as name, title, coefficient, credits, and workload attributes. They must also implement average and credit calculations.

A `Module` stores grades and evaluation rules, whereas a `Unit` aggregates multiple modules.

Q6. Explain the inheritance relationship between Unit and Module.

Answer:

A `Unit` should not inherit from `Module`, and a `Module` should not inherit from `Unit`, because a unit is not a module and a module is not a unit.

Both classes inherit from the common abstract parent `AcademicElement`. The relationship between `Unit` and `Module` is containment or aggregation.

Q7. Why use abstract `AcademicElement` and test the code?

Answer:

The `AcademicElement` class centralizes common attributes and declares `calculate_average()` and `calculate_credits()` as abstract methods.

A subclass must implement these methods. In Python, the class must inherit from `ABC` and import `abstractmethod` from the `abc` module to enforce the abstract-method requirements.

Screenshot (optional):

Q8. Can Semester be a subclass under the abstraction principle?

Answer:

Yes. A `Semester` is also an `AcademicElement`, so it can inherit from `AcademicElement` and implement its own average and credit calculation methods. A semester contains units.

Q9. Check encapsulation in Module and modify it if necessary.

Answer:

The original `_grades` dictionary is protected by convention, rather than strictly private.

The `set_grade()` method controls grade updates, and the average calculation method reads the grades internally.

The corrected implementation validates grades between 0 and 20 and raises a clear error if required grades have not been supplied, instead of failing with a `None` arithmetic error.

Q10. Override `calculate_average()` and `calculate_credits()` for Unit.

Answer:

The unit average is the coefficient-weighted mean of its module averages:

$$\text{Unit Average} = \frac{\sum_{i=1}^{n} (\text{Module Average}_i \times \text{Module Coefficient}_i)}{\sum_{i=1}^{n} \text{Module Coefficient}_i}.$$

If the unit average is at least 10, all credits of its modules are counted. Otherwise, only the credits earned by modules individually are counted. An empty unit returns an average of zero.

Q11. Define Semester and implement its methods.

Answer:

The `Semester` class contains units. Its average is calculated as the coefficient-weighted mean of the unit averages. Its credits are the sum of the credits earned by its units.

This implementation assumes that each unit's coefficient is the sum of its module coefficients.

Screenshot and code link (optional):

Q12. Implement the whole GSI Canvas logic.

Answer:

Implement the main application logic in `main.py`. The program should integrate the academic elements, manage students, and test the interaction between the different classes.

The implementation should allow students to be added and should verify that the academic model works correctly.

Main program placeholder:

```
# main.py
# Implement the complete GSI Canvas logic here.
# Add students and test the academic model.
```